

**UNIVERSIDAD CATÓLICA DE SANTA MARÍA
ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS**

LABORATORIO 09:**JAVA SWING BÁSICO: INTRODUCCIÓN Y CONTROLES****I****OBJETIVOS**

- Conocer qué son las Interfaces Gráficas de Usuario y como construirlas a través de componentes.
- Implementar ejemplos de diferentes componentes usados en las Interfaces Gráficas Usuario
- Utilizar y manejar controles Swing

II**TEMAS A TRATAR**

- Interfaz Gráfica de Usuario GUI
- AWT y en qué se diferencia con Swing
- Componentes de Swing para crear GUIs
- Manejo de Eventos e Interfaces de Escucha
- Clases Adaptadoras
- Manejo de Eventos de Teclas
- Administrador de esquemas
- Uso de paneles

III**MARCO TEÓRICO****Interfaz Gráfica De Usuario (GUI)**

Una interfaz gráfica de usuario (GUI) ofrece un mecanismo amigable para que el usuario interactúe con una aplicación. Una GUI dota a una aplicación con una “apariencia visual” única. Las GUI se construyen a partir de los componentes de GUI. Un componente de GUI es un objeto con el que el usuario interactúa a través del ratón, del teclado o de otra forma de entrada. **Para este caso utilizaremos componentes GUI de Swing** del paquete `javax.swing`.

Entrada/Salida simple basada en GUI con `JOptionPane`

La mayoría de las aplicaciones que usamos a diario utilizan ventanas o cuadros de diálogo (también conocidos como diálogos) para interactuar con el usuario. Los cuadros de diálogo son ventanas en las cuales los programas muestran mensajes importantes al usuario, u obtienen información de éste. La clase **`JOptionPane`** de Java (paquete **`javax.swing`**) proporciona cuadros de diálogo prefabricados para entrada y salida. Estos diálogos se muestran mediante la invocación de los métodos static de `JOptionPane`.

A continuación se muestra un ejemplo básico del uso de la clase `JOptionPane`

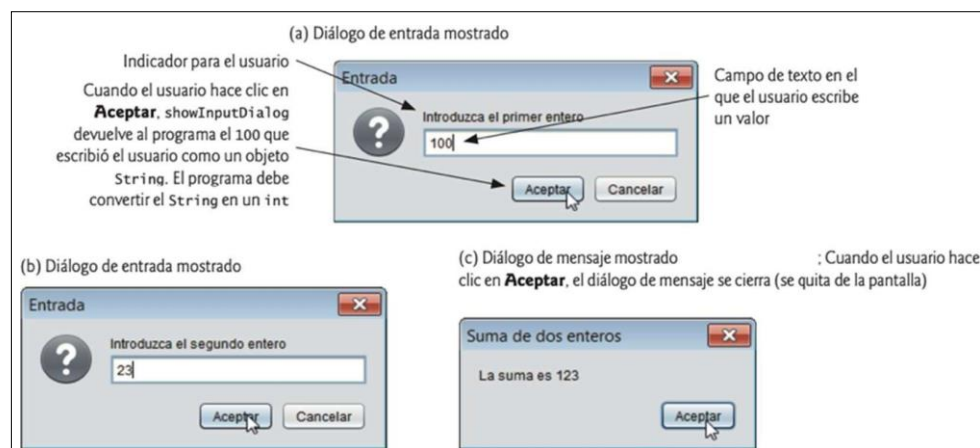
```
import javax.swing.JOptionPane;

public class App {
    public static void main(String[] args) throws Exception {
        // obtiene la entrada del usuario de los diálogos de entrada de JOptionPane
        String primerNumero =
            JOptionPane.showInputDialog("Introduzca el primer entero");
        String segundoNumero =
            JOptionPane.showInputDialog("Introduzca el segundo entero");

        // convierte las entradas String en valores int para usarlos en un cálculo
        int numero1 = Integer.parseInt(primerNumero);
        int numero2 = Integer.parseInt(segundoNumero);
        int suma = numero1 + numero2;

        // muestra los resultados en un diálogo de mensajes de JOptionPane
        JOptionPane.showMessageDialog(null, "La suma es " + suma, "Suma de dos enteros",
            JOptionPane.PLAIN_MESSAGE);
    }
}
```

Un ejemplo de lo mostrado por el programa si lo ejecutamos sería el siguiente:



Revisando la secuencia de lo realizado por el programa se tiene lo siguiente:

- 1) Usando `JOptionPane.showInputDialog(...)` Se abre un cuadro de dialogo solicitando al usuario que ingrese un 1er valor y se acepta el valor ingresado
- 2) Se hace lo mismo que el paso 1 para ingresar un 2do valor
- 3) Los valores son almacenados como variables String por lo que se convierten estos valores a tipo entero.
- 4) Se suman los valores ingresados
- 5) Usando `JOptionPane.showMessageDialog (...)` se muestra en un cuadro de diálogo el valor de la suma de los mismos. El 2do argumento del método `showMessageDialog (...)` se utiliza para indicar si se mostrará algún ícono que indique el tipo de importancia que tiene el mensaje a mostrar. En este caso se utiliza la constante `JOptionPane.PLAIN_MESSAGE` para indicar que no se muestre ningún ícono.

A continuación, se muestra los posibles valores que podría tener el 2do argumento.

Tipo de diálogo de mensaje	Icono	Descripción
ERROR_MESSAGE		Indica un error.
INFORMATION_MESSAGE		Indica un mensaje informativo.
WARNING_MESSAGE		Advierte al usuario sobre un problema potencial.
QUESTION_MESSAGE		Hace una pregunta al usuario. Por lo general, este diálogo requiere una respuesta, como hacer clic en un botón Sí o No .
PLAIN_MESSAGE	sin icono	Un diálogo que contiene un mensaje, pero no un icono.

Generalidades de los componentes de Swing

Aunque es posible realizar operaciones de entrada y salida utilizando los diálogos de `JOptionPane`, la mayoría de las aplicaciones de GUI requieren interfaces de usuario más elaboradas que permiten a los desarrolladores de aplicaciones crear GUI robustas. A continuación, se muestra una lista de varios de los componentes básicos de la GUI de Swing que vamos a analizar.

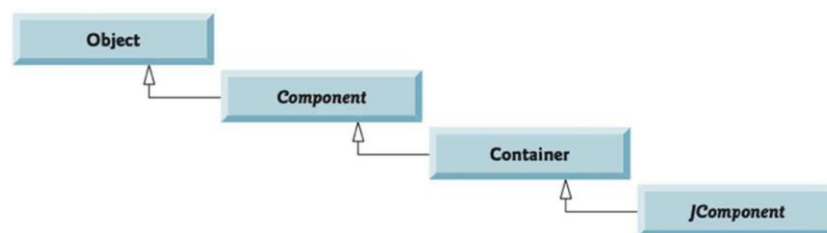
Componente	Descripción
<code>JLabel</code>	Muestra <i>texto</i> o iconos que <i>no pueden editarse</i> .
<code>TextField</code>	Por lo general <i>recibe entrada</i> del usuario.
<code>Button</code>	Activa un evento cuando se oprime mediante el ratón.
<code>CheckBox</code>	Especifica una opción que puede <i>seleccionarse</i> o <i>no seleccionarse</i> .
<code>ComboBox</code>	Una <i>lista desplegable de elementos</i> , a partir de los cuales el <i>usuario</i> puede realizar una <i>selección</i> .
<code>List</code>	Una <i>lista de elementos</i> a partir de los cuales el usuario puede realizar una <i>selección</i> , <i>haciendo clic</i> en <i>cualquiera</i> de ellos. <i>Pueden seleccionarse varios</i> elementos.
<code>Panel</code>	Un área en la que pueden <i>colocarse</i> y <i>organizarse</i> los <i>componentes</i> .

AWT y en qué se diferencia con Swing

En realidad, hay dos conjuntos de componentes de GUI en Java. En los primeros días de Java, las GUI se creaban a partir de componentes del Abstract Window Toolkit (AWT) en el paquete `java.awt`. Éstos se ven como los componentes de GUI nativos de la plataforma en la que se ejecuta un programa de Java. Por ejemplo, un objeto de tipo `Button` que se muestra en un programa de Java ejecutándose en Microsoft Windows tendrá la misma apariencia que los botones en las demás aplicaciones Windows. En el sistema operativo Apple Mac OS X, el objeto `Button` tendrá la misma apariencia visual que los botones en las demás aplicaciones Mac.

- Los componentes de la GUI de Swing nos permiten especificar una apariencia visual uniforme para nuestras aplicaciones en todas las plataformas, o usar la apariencia visual personalizada en cada plataforma.
- La mayoría de los componentes de Swing son componentes ligeros; es decir, que se escriben, manipulan y visualizan por completo en Java. Los componentes de AWT son componentes pesados, ya que dependen del sistema de ventanas de la plataforma local para determinar su funcionalidad y su apariencia visual.

Jerarquía de Clases de la Clase Componentes GUI



La clase ***Component*** (*paquete `java.awt`*) es una superclase que declara las características comunes de los componentes de GUI en los paquetes *`java.awt`* y *`javax.swing`*.

Cualquier objeto que sea un ***Container*** (*paquete `java.awt`*) se puede utilizar para organizar a otros objetos *Component*, adjuntando esos objetos *Component* al objeto *Container*. Los objetos *Container* se pueden colocar en otros objetos *Container* para organizar una GUI.

La clase ***JComponent*** (*paquete `javax.swing`*) es una subclase de *Container*. *JComponent* es la superclase de todos los componentes ligeros de Swing, y declara los atributos y comportamientos comunes. Debido a que *JComponent* es una subclase de *Container*, todos los componentes ligeros de Swing son también objetos *Container*. Algunas de las características comunes que soporta *JComponent* son:

- 1) Una apariencia visual adaptable, la cual puede utilizarse para personalizar la apariencia de los componentes (por ejemplo, para usarlos en plataformas específicas).
- 2) Teclas de método abreviado (llamadas nemónicos) para un acceso directo a los componentes de la GUI por medio del teclado.
- 3) Breves descripciones del propósito de un componente de la GUI (lo que se conoce como cuadros de información sobre herramientas o tool tips) que se muestran cuando el cursor

del ratón se coloca sobre el componente durante un breve periodo.

- 4) Soporte de accesibilidad para personas con discapacidad o mostrar información en distintos idiomas

Presentación de texto e imágenes en una ventana

La mayoría de las ventanas que creará y que pueden contener componentes de GUI de Swing son una instancia de la clase **JFrame** o una subclase de **JFrame**. Ésta es una subclase indirecta de la clase **java.awt.Window** que proporciona los atributos y comportamientos básicos de una ventana, como una barra de título en la parte superior, y botones para minimizar, maximizar y cerrar la ventana. Como la GUI de una aplicación por lo general es específica para esa aplicación, la mayoría de los ejemplos mostrados luego consistirán en dos clases:

- a) Una subclase de **JFrame** que nos ayuda a demostrar los nuevos conceptos de la GUI
- b) Una clase de aplicación, en la que *main* crea y muestra la ventana principal de la aplicación.

En esta guía de laboratorio se busca dar una idea inicial de cómo manejar los principales componentes Swing. Para tener información detallada y completa del manejo de otros componentes y sus métodos y atributos asociados visite: <https://docs.oracle.com/javase/7/docs/api/javax/swing/JComponent.html>

Especificando el Esquema

Al crear una GUI, cada componente de ésta debe adjuntarse a un contenedor, como una ventana creada con un objeto **JFrame**. Además, por lo general debemos decidir en dónde colocar cada componente de la GUI y establecer su tamaño; esto se conoce como *Especificar el Esquema*. Java cuenta con varios administradores de esquemas que pueden ayudarle a colocar los componentes. Así mismo, hay varios IDEs que permiten realizar el mismo trabajo de un administrador de esquema sólo utilizando el ratón y de manera visual.

A continuación, utilizaremos un administrador de esquemas de Java a nivel de código.

En el administrador de esquemas **FlowLayout**, los componentes de la GUI se colocan en un contenedor de izquierda a derecha, en el orden en el que el programa los une al contenedor. Cuando no hay más espacio para acomodar los componentes en la línea actual, se siguen mostrando de izquierda a derecha en la siguiente línea. Si se cambia el tamaño del contenedor, un esquema **FlowLayout** reordena los componentes usando menos o más filas con base en la nueva anchura del contenedor.

```
// Archivo "App.java"
// Prueba de LabelFrame
import javax.swing.JFrame;

public class App
{
    public static void main(String[] args)
    {
        LabelFrame marcoEtiqueta = new LabelFrame();
        marcoEtiqueta.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        marcoEtiqueta.setSize(260, 180);
        marcoEtiqueta.setVisible(true);
    }
} // Fin de App Prueba Label

// Archivo "LabelFrame.java"
// Componentes JLabel con texto e iconos.
import java.awt.FlowLayout; // especifica cómo se van a ordenar los componentes
import javax.swing.JFrame; // proporciona las características básicas de una ventana
import javax.swing.JLabel; // muestra texto e imágenes
import javax.swing.SwingConstants; // constantes comunes utilizadas con Swing
import javax.swing.Icon; // interfaz utilizada para manipular imágenes
```

```

import javax.swing.ImageIcon; // carga las imágenes

public class LabelFrame extends JFrame
{
    private JLabel etiqueta1; // JLabel sólo con texto
    private JLabel etiqueta2; // JLabel construida con texto y un icono
    private JLabel etiqueta3; // JLabel con texto adicional e icono 15

    // El constructor de LabelFrame agrega objetos JLabel a JFrame
    public LabelFrame()
    {
        // Constructor de JLabel con un argumento String
        super("Prueba de JLabel");
        setLayout(new FlowLayout()); // establece el esquema del marco

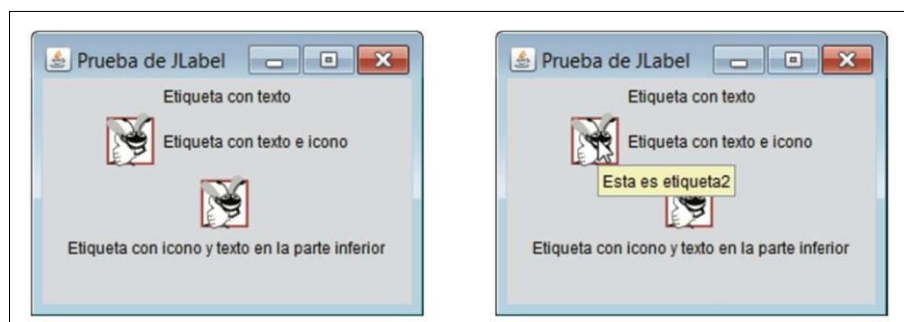
        // Constructor de JLabel con un argumento String
        etiqueta1 = new JLabel("Etiqueta con texto");
        etiqueta1.setToolTipText("Esta es etiqueta1");
        add(etiqueta1); // agrega etiqueta1 a JFrame

        // Constructor de JLabel con argumentos de cadena, Icono y alineación
        Icon insecto = new ImageIcon(getClass().getResource("insecto1.png"));
        etiqueta2 = new JLabel("Etiqueta con texto e icono", insecto,
                               SwingConstants.LEFT);
        etiqueta2.setToolTipText("Esta es etiqueta2");
        add(etiqueta2); // agrega etiqueta2 a JFrame

        etiqueta3 = new JLabel(); // constructor de JLabel sin argumentos
        etiqueta3.setText("Etiqueta con icono y texto en la parte inferior");
        etiqueta3.setIcon(insecto); // agrega icono a JLabel
        etiqueta3.setHorizontalTextPosition(SwingConstants.CENTER);
        etiqueta3.setVerticalTextPosition(SwingConstants.BOTTOM);
        etiqueta3.setToolTipText("Esta es etiqueta3");
        add(etiqueta3); // agrega etiqueta3 a JFrame
    }
} // fin de la clase LabelFrame

```

Un ejemplo de lo mostrado por el programa si lo ejecutamos sería el siguiente:



Revisando la secuencia de lo realizado por el programa se tiene lo siguiente:

En el archivo “*LabelFrame.java*”

- 1) Se especifica el esquema para el componente *LabelFrame* que hemos creado. Para eso se crea objeto de clase *FlowLayout* y lo asignamos al *LabelFrame* con el método *setLayout*.

- 2) Creamos un objeto *JLabel* para nuestra Etiqueta 1, le cargamos el texto y su mensaje alternativo a través de su constructor y método *setToolTipText* respectivamente. Luego lo añadimos a nuestro componente *LabelFrame*.
- 3) Para crear nuestra Etiqueta 2 hacemos algo similar al paso 2) pero con la diferencia que además agregaremos una imagen utilizando los componentes: interfaz *Icon* para manipular imágenes, *ImageIcon* para cargar imágenes y *SwingConstants* utilizando su constante *CENTER* para establecer una posición del texto con relación a la imagen.

Constante	Descripción	Constante	Descripción
<i>Constantes de posición horizontal</i>		<i>Constantes de posición vertical</i>	
LEFT	Coloca el texto a la izquierda	TOP	Coloca el texto en la parte superior
CENTER	Coloca el texto en el centro	CENTER	Coloca el texto en el centro
RIGHT	Coloca el texto a la derecha	BOTTOM	Coloca el texto en la parte inferior

Importante: El archivo de imagen (.GIF , .PNG, .JPEG) debe estar situado en la carpeta de nuestro proyecto donde se crean los archivos *.class* generados desde los *.java* ya que esa es la ruta predeterminada para usar los recursos para el proyecto.

- 4) Para nuestra Etiqueta 3 hacemos algo similar al paso 3) pero utilizando los métodos de la clase *JLabel*.

En el archivo "*App.java*"

- 5) Se crea y se muestra una ventana de tipo *LabelFrame*. Para eso se utiliza el método *setDefaultCloseOperation* para establecer la acción predeterminada cuando cerramos la ventana. En este caso se pasa como argumento la constante *JFrame.EXIT_ON_CLOSE* para indicar que el programa debe terminar. Así mismo, se establece el tamaño de la ventana (ancho y alto respectivamente) con el método *setSize* y se establece que de manera predeterminada estará visible con el método *setVisible*.

Campos de texto y una introducción al manejo de eventos con clases anidadas

Por lo general, un usuario interactúa con la GUI de una aplicación para indicar las tareas que ésta debe realizar. Por ejemplo, cuando presionamos un botón en una aplicación y esta realiza algún proceso o acción y nos devuelve un resultado.

Las GUI son controladas por eventos. Cuando el usuario interactúa con un componente de la GUI, la interacción (conocida como un evento) controla el programa para que realice una tarea.

Algunas interacciones o eventos comunes del usuario que podrían hacer que una aplicación realizara una tarea incluyen:

- El hacer clic en un botón
- Escribir en un campo de texto
- Seleccionar un elemento de un menú
- Cerrar una ventana
- Mover el ratón.

El código que realiza una tarea en respuesta a un evento se llama *manejador de eventos* y al proceso en general de responder a los eventos se le conoce como manejo de eventos.

En el siguiente ejemplo se utiliza las clases *JTextField* y *JPasswordField* para crear y manipular

cuatro campos de texto. Cuando el usuario escribe en uno de los campos de texto y después oprime *Intro*, la aplicación muestra un cuadro de diálogo de mensaje que contiene el texto que escribió el usuario. Así mismo, cuando el usuario oprime *Intro* en el objeto `JPasswordField`, se revela la contraseña.

```
// Archivo "CampoTextoMarco.java"
// Los componentes JTextField y JPasswordField.
import java.awt.FlowLayout;
import java.awt.event.ActionListener;
import java.awt.event.ActionEvent;
import javax.swing.JFrame;
import javax.swing.JTextField;
import javax.swing.JPasswordField;
import javax.swing.JOptionPane;

public class CampoTextoMarco extends JFrame {
    private final JTextField campoTexto1; // campo de texto con tamaño fijo
    private final JTextField campoTexto2; // campo de texto con texto
    private final JTextField campoTexto3; // campo de texto con texto y tamaño
    private final JPasswordField campoContrasenia; // campo de contraseña con texto

    // El constructor de CampoTextoMarco agrega objetos JTextField a JFrame
    public CampoTextoMarco()
    {

        super("Prueba de JTextField y JPasswordField");
        setLayout(new FlowLayout());

        // construye campo de texto con 10 columnas
        campoTexto1 = new JTextField(10);
        add(campoTexto1); // agrega campoTexto1 a JFrame

        // construye campo de texto con texto predeterminado
        campoTexto2 = new JTextField("Escriba el texto aqui");
        add(campoTexto2); // agrega campoTexto2 a JFrame

        // construye campo de texto con texto predeterminado y 21 columnas
        campoTexto3 = new JTextField("Campo de texto no editable", 21);
        campoTexto3.setEditable(false); // deshabilita la edición
        add(campoTexto3); // agrega campoTexto3 a JFrame

        // construye campo de contraseña con texto predeterminado
        campoContrasenia = new JPasswordField("Texto oculto");
        add(campoContrasenia); // agrega campoContrasenia a JFrame

        // registra los manejadores de eventos
        ManejadorCampoTexto manejador = new ManejadorCampoTexto();
        campoTexto1.addActionListener(manejador);
        campoTexto2.addActionListener(manejador);
        campoTexto3.addActionListener(manejador);
        campoContrasenia.addActionListener(manejador);
    }

    // clase interna privada para el manejo de eventos
    private class ManejadorCampoTexto implements ActionListener
    {
        // procesa los eventos de campo de texto
        @Override
        public void actionPerformed(ActionEvent evento)
```

```

    {
        String cadena = "";

        // el usuario oprimió Intro en el objeto JTextField campoTexto1
        if (evento.getSource() == campoTexto1)
            cadena = String.format("campoTexto1: %s", evento.getActionCommand());

        // el usuario oprimió Intro en el objeto JTextField campoTexto2
        else if (evento.getSource() == campoTexto2)
            cadena = String.format("campoTexto2: %s", evento.getActionCommand() );

        // el usuario oprimió Intro en el objeto JTextField campoTexto3
        else if (evento.getSource() == campoTexto3)
            cadena = String.format("campoTexto3: %s", evento.getActionCommand());

        // el usuario oprimió Intro en el objeto JTextField campoContrasenia
        else if (evento.getSource() == campoContrasenia)
            cadena = String.format("campoContrasenia: %s",
                                   evento.getActionCommand());

        // muestra el contenido del objeto JTextField
        JOptionPane.showMessageDialog(null, cadena);
    }
} //fin de la clase privada interna ManejadorCampoTexto
} //fin de la clase CampoTextoMarco

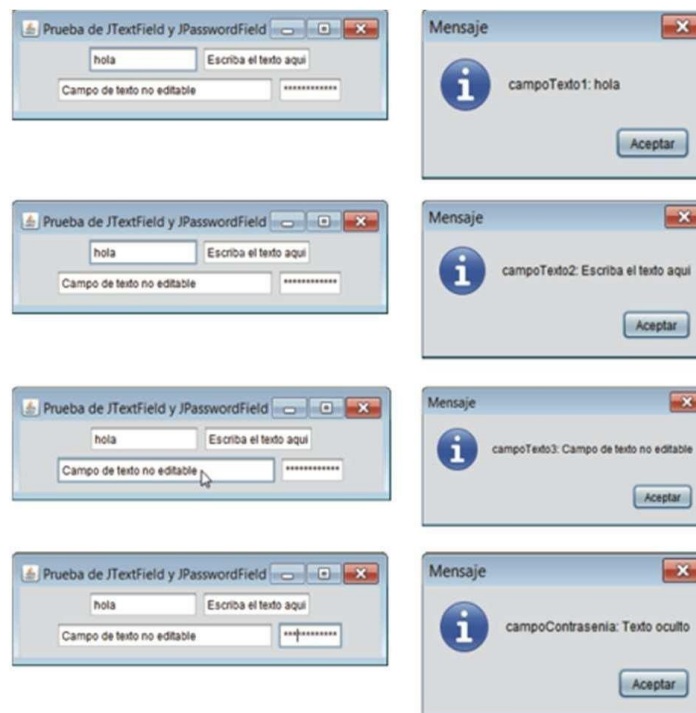
// Prueba de CampoTextoMarco
import javax.swing.JFrame;

public class App
{
    public static void main(String[] args) {
        CampoTextoMarco campoTextoMarco = new CampoTextoMarco();
        campoTextoMarco.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        campoTextoMarco.setSize(350, 100);
        campoTextoMarco.setVisible(true);
    }
} // fin de la App CampoTextoMarco

```

Un ejemplo de lo mostrado por el programa si lo ejecutamos sería el siguiente:





Revisando la secuencia de lo realizado por el programa se tiene lo siguiente:

- 1) Se crea la GUI estableciendo el esquema del objeto *CampoTextoMarco* a *FlowLayout*. Luego se crea 3 objetos de tipo *JTextField* y 1 de tipo *JPasswordField* estableciendo un valor inicial y/o estableciendo el ancho máximo del control en columnas a través de sus constructores. Para algunos casos se establece si el objeto no podrá editarse.
- 2) Se hace uso de una clase anidada para implementar un *manejador de eventos*, para nuestro caso se establece un *manejador de eventos* para los componentes GUI a través de la clase anidada *ManejadorCampoTexto*. Para nuestro ejemplo este *manejador de eventos* deberá mostrar un diálogo de mensaje que contenga el texto de un campo de texto, cuando el usuario oprime *Intro* en ese campo. Para esto necesitaremos:
 - a. Crear una clase que represente al manejador de eventos e implemente una interfaz apropiada, conocida como **interfaz de escucha de eventos**. Para este ejemplo la Interfaz *ActionListener*
 - b. Indicar que se debe **notificar a un objeto de la clase** del paso a. **cuando ocurra el evento**. A esto se le conoce como *registrar el manejador de eventos*. Para este ejemplo sobrescribimos el método *actionPerformed*

Java nos permite declarar clases dentro de otras clases; a éstas se les conoce como **clases anidadas**. Las *clases anidadas* pueden ser *static* o no *static*. Las clases anidadas no *static* se llaman **clases internas**, y se utilizan con frecuencia para implementar *manejadores de eventos*.

Un objeto de una *clase interna* debe crearse mediante un objeto de la **clase de nivel superior** que contenga a la clase interna. Cada objeto de la *clase interna* tiene implícitamente una referencia a un objeto de su *clase de nivel superior*. El objeto de la *clase interna* puede usar esta referencia implícita para acceder

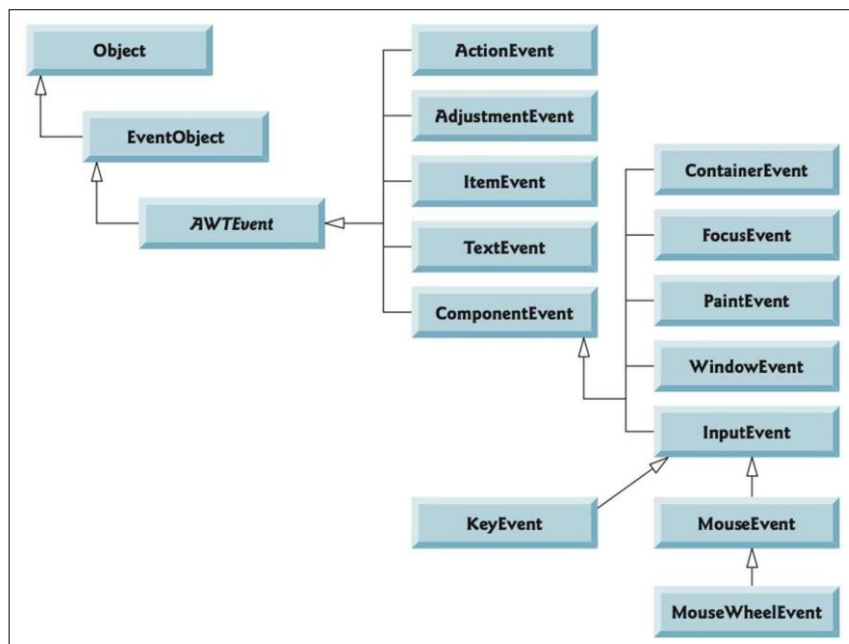
directamente a todas las variables y métodos de la clase de nivel superior.

- 3) Definiendo el contenido de la clase interna `ManejadorCampoTexto`. El método `actionPerformed` (único método de la interfaz `ActionListener`) especifica las tareas a realizar cuando ocurre un evento `ActionEvent`. Para este caso identifica que tipo de objeto activó al evento, para esto utiliza el método `getSource` de `ActionEvent`; a continuación, obtiene el texto del objeto de campo de texto identificado a través del método `getActionCommand`, finalmente muestra el texto en una ventana de diálogo.
- 4) Registramos el manejador de eventos para cada campo de texto. En el constructor de `CampoTextoMarco`, se crea un objeto `ManejadorCampoTexto` y lo asigna a la variable manejador. Luego el programa debe registrar este objeto como el manejador de eventos para nuestros campos de texto `TextField` y el objeto `PasswordField`.

Tipos de eventos comunes de la GUI e interfaces de escucha

Además del evento, que ocurre cuando el usuario oprime *Intro* en un campo de texto el cual se almacena en un objeto `ActionEvent`, *pueden* ocurrir muchos tipos distintos de eventos cuando el usuario interactúa con una GUI. La información acerca de cualquier evento se almacena en un objeto de una clase que extiende a `AWTEvent` (del paquete `java.awt`).

A continuación, se muestra una jerarquía que contiene muchas clases de eventos del paquete `java.awt.event`. Estos tipos de eventos se utilizan tanto con componentes de AWT como de Swing. Los tipos de eventos adicionales que son específicos para los componentes de GUI de Swing se declaran en el paquete `javax.swing.event`.



Resumiendo, las tres partes requeridas para el mecanismo de manejo de eventos son:

- 1) El origen del evento: asociado a un componente que es el que la genera
- 2) El objeto del evento: asociado al tipo de acción realizada (presión de tecla, movimiento de mouse, ingreso de texto, selección de ítem, etc.)

- 3) El componente de escucha del evento: asociado al objeto que dependiendo del tipo evento que reciba invocará a uno de sus métodos para realizar alguna acción.

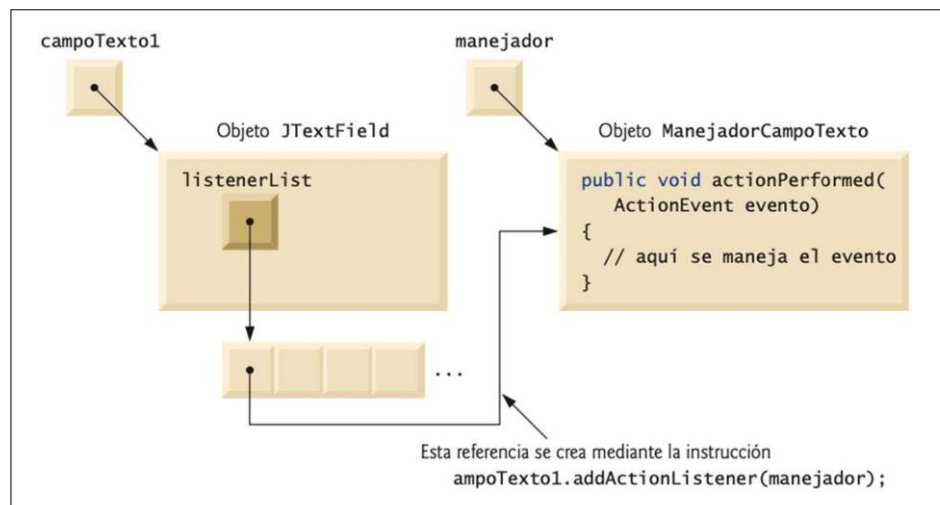
A este modelo de manejo de eventos se le conoce como **modelo de eventos por delegación**, ya que el procesamiento de un evento se delega a un objeto específico (el componente de escucha de eventos) de la aplicación.

- Para cada tipo de objeto evento, hay por lo general una interfaz de escucha de eventos que le corresponde.
- Un componente de escucha de eventos para un evento de GUI es un objeto de una clase que implementa a una o más de las interfaces de escucha de eventos de los paquetes *java.awt.event* y *javax.swing.event*.

Cómo funciona el manejo de eventos

Registro de Eventos

Todo *JComponent* tiene una variable de instancia llamada *listenerList*, que hace referencia a un objeto de la clase *EventListenerList* (paquete *javax.swing.event*). Cada objeto de una subclase de *JComponent* mantiene referencias a todos sus componentes de escucha registrados en *listenerList*. A continuación, se muestra un diagrama de cómo se realiza el registro de eventos considerando el ejemplo anterior donde se usó componentes de tipo *TextField*



Aunque no se muestra en el diagrama, una nueva entrada en el objeto *listenerList* también incluye un tipo del componente de escucha (para nuestro ejemplo el *ActionListener*). Mediante el uso de este mecanismo, cada componente ligero de GUI de Swing mantiene su propia lista de componentes de escucha que se registraron para manejar los eventos del componente.

Invocación al manejador de eventos

Todo componente de la GUI soporta varios tipos de eventos, incluyendo *eventos de ratón*, *eventos de tecla* y *otros* más. Cuando ocurre un evento, éste se despacha solamente a los componentes de escucha de eventos del tipo apropiado. El despachamiento (**dispatching**) es simplemente el proceso por el cual el componente de la GUI llama a un método manejador de eventos en cada uno de sus componentes de escucha registrados para el tipo de evento que ocurrió.

Cada tipo de evento tiene una o más interfaces de escucha de eventos correspondientes. Por ejemplo, los eventos tipo *ActionEvent* son manejados por objetos *ActionListener*, los eventos tipo *MouseEvent* son manejados por objetos *MouseListener* y *MouseMotionListener*, y los eventos tipo *KeyEvent* son manejados por objetos *KeyListener*.

Cuando ocurre un evento, el componente de la GUI recibe (de la JVM) un ID de evento único, el cual especifica el tipo de evento. El componente de la GUI utiliza el ID de evento para decidir a cuál tipo de componente de escucha debe despacharse el evento, y para decidir cuál método llamar en cada objeto de escucha.

Ejemplos:

- a) Para un *ActionEvent*, el evento se despacha al método *actionPerformed* de todos los objetos *ActionListener* registrados (el único método en la interfaz *ActionListener*).
- b) En el caso de un *MouseEvent*, el evento se despacha a todos los objetos *MouseListener* o *MouseMotionListener* registrados, dependiendo del evento de ratón que ocurra.

Manejo de Eventos de Ratón

En esta sección presentaremos las interfaces de escucha de eventos ***MouseListener*** y ***MouseMotionListener*** para manejar eventos de ratón. Estos eventos pueden procesarse para cualquier componente de la GUI que se derive de *java.awt.Component*. El paquete *javax.swing.event* contiene la interfaz ***MouseInputListener***, la cual extiende a las interfaces *MouseListener* y *MouseMotionListener* para crear una sola interfaz que contiene todos los métodos de *MouseListener* y *MouseMotionListener*. Estos métodos se llaman cuando el ratón interactúa con un objeto *Component*, si se registran objetos componentes de escucha de eventos apropiados para ese objeto *Component*.

Los métodos de las interfaces *MouseListener* y *MouseMotionListener* se muestran a continuación:

Métodos de las interfaces *MouseListener* y *MouseMotionListener*

Métodos de la interfaz *MouseListener*

```
public void mousePressed(MouseEvent evento)
```

Es llamado cuando se *oprime* un botón del ratón, mientras el cursor del ratón está sobre un componente.

```
public void mouseClicked(MouseEvent evento)
```

Es llamado cuando se *oprime y suelta* un botón del ratón, mientras el cursor del ratón permanece estacionario sobre un componente. Este evento siempre va precedido por una llamada a *mousePressed* y *mouseReleased*.

```
public void mouseReleased(MouseEvent evento)
```

Es llamado cuando se *suelta un botón de ratón después de ser oprimido*. Este evento siempre va precedido por una llamada a *mousePressed* y por una o más llamadas a *mouseDragged*.

```
public void mouseEntered(MouseEvent evento)
```

Es llamado cuando el cursor del ratón *entra* a los límites de un componente.

```
public void mouseExited(MouseEvent evento)
```

Es llama cuando el cursor del ratón *sale* de los límites de un componente.

Métodos de la interfaz MouseMotionListener

```
public void mouseDragged(MouseEvent evento)
```

Es llamado cuando el botón del ratón se *oprime* mientras el cursor del ratón se encuentra sobre un componente y el ratón se *mueve* mientras el botón *sigue oprimido*. Este evento siempre va precedido por una llamada a `mousePressed`. Todos los eventos de arrastre del ratón se envían al componente en el cual empezó la acción de arrastre.

```
public void mouseMoved(MouseEvent evento)
```

Es llamado al *moverse* el ratón (sin oprimir los botones del ratón) cuando su cursor se encuentra sobre un componente. Todos los eventos de movimiento se envían al componente sobre el cual se encuentra el ratón posicionado en ese momento.

Java también cuenta con la interfaz ***MouseWheelListener*** para permitir a las aplicaciones responder a la rotación de la rueda de un ratón. Esta interfaz declara el método ***mouseWheelMoved***, el cual recibe un evento ***MouseWheelEvent*** como argumento. La clase ***MouseWheelEvent*** (una subclase de ***MouseEvent***) contiene métodos que permiten al manejador de eventos obtener información acerca de la cantidad de rotación de la rueda.

Ejemplo de cómo rastrear eventos de ratón en un objeto JPanel

La siguiente aplicación *RastreadorRaton* demuestra el uso de los métodos de las interfaces ***MouseListener*** y ***MouseMotionListener***. La clase de manejador de evento implementa ambas interfaces, por lo que se declaran los siete métodos de estas dos interfaces. Cada evento de ratón en este ejemplo muestra un objeto *String* en el objeto *JLabel* llamado *barraEstado*, que está unido

a la parte inferior de la ventana.

```
// Archivo "MarcoRastreadorRaton.java"
// Manejo de eventos de ratón.
import java.awt.Color;
import java.awt.BorderLayout;
import java.awt.event.MouseListener;
import java.awt.event.MouseMotionListener;
import java.awt.event.MouseEvent;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JPanel;

public class MarcoRastreadorRaton extends JFrame {
    private final JPanel panelRaton; // panel en el que ocurrirán los eventos de ratón
    private final JLabel barraEstado; // muestra información de los eventos

    // El constructor de MarcoRastreadorRaton establece la GUI y
    // registra los manejadores de eventos de ratón
    public MarcoRastreadorRaton()
    {
        super("Demostracion de los eventos de raton");

        panelRaton = new JPanel();
        panelRaton.setBackground(Color.WHITE);
        add(panelRaton, BorderLayout.CENTER); // agrega el panel a JFrame

        barraEstado = new JLabel("Raton fuera de JPanel");
        add(barraEstado, BorderLayout.SOUTH); // agrega etiqueta a JFrame

        // crea y registra un componente de escucha para los eventos de ratón y de su
        // movimiento
        ManejadorRaton manejador = new ManejadorRaton();
        panelRaton.addMouseListener(manejador);
        panelRaton.addMouseMotionListener(manejador);
    }

    private class ManejadorRaton implements MouseListener, MouseMotionListener
    {
        // Los manejadores de eventos de MouseListener
        // manejan el evento cuando se suelta el ratón justo después de oprimir el botón
        @Override
        public void mouseClicked(MouseEvent evento)
        {
            barraEstado.setText(String.format("Se hizo clic en [%d, %d]", evento.getX(),
                evento.getY()));
        }

        // maneja evento cuando se oprime el ratón
        @Override
        public void mousePressed(MouseEvent evento)
        {
            barraEstado.setText(String.format("Se oprimio en [%d, %d]", evento.getX(),
                evento.getY()));
        }

        // maneja evento cuando se suelta el botón del ratón
        @Override
        public void mouseReleased(MouseEvent evento)
    }
```



```

        barraEstado.setText(String.format("Se solto en [%d, %d]", evento.getX(),
            evento.getY()));
    }

    // maneja evento cuando el ratón entra al área
    @Override
    public void mouseEntered(MouseEvent evento)
    {
        barraEstado.setText(String.format("Raton entro en [%d, %d]", evento.getX(),
            evento.getY()));
        panelRaton.setBackground(Color.GREEN);
    }

    // maneja evento cuando el ratón sale del área
    @Override
    public void mouseExited(MouseEvent evento)
    {
        barraEstado.setText("Raton fuera de JPanel");
        panelRaton.setBackground(Color.WHITE);
    }

    // Los manejadores de eventos de MouseMotionListener manejan
    // el evento cuando el usuario arrastra el ratón con el botón oprimido
    @Override
    public void mouseDragged(MouseEvent evento)
    {
        barraEstado.setText(String.format("Se arrastro en [%d, %d]", evento.getX(),
            evento.getY()));
    }

    // maneja evento cuando el usuario mueve el ratón
    @Override
    public void mouseMoved(MouseEvent evento)
    {
        barraEstado.setText(String.format("Se movio en [%d, %d]", evento.getX(),
            evento.getY()));
    }
} // fin de la clase interna ManejadorRaton
} // fin de la clase MarcoRastreadorRaton

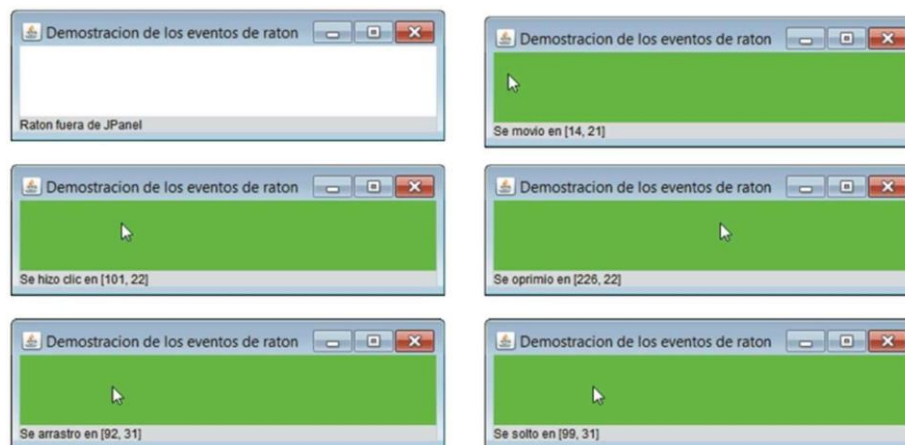
///// PRUEBA DE Eventos de Raton /////
// Prueba de MarcoRastreadorBoton

import javax.swing.JFrame;

public class App
{
    public static void main(String[] args)
    {
        MarcoRastreadorRaton marcoRastreadorRaton = new MarcoRastreadorRaton();
        marcoRastreadorRaton.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        marcoRastreadorRaton.setSize(300, 100);
        marcoRastreadorRaton.setVisible(true);
    }
} // fin de la clase Rastreador Raton

```

Un ejemplo de lo mostrado por el programa si lo ejecutamos sería el siguiente:



Como vimos, por lo general debemos especificar el esquema de los componentes de GUI en un objeto *JFrame*. En esa sección presentamos el administrador de esquemas *FlowLayout*. Aquí utilizamos el esquema predeterminado del panel de contenido de un objeto *JFrame*, ***BorderLayout***, el cual ordena los componentes en cinco regiones: *NORTH*, *SOUTH*, *EAST*, *WEST* y *CENTER*. *NORTH* corresponde a la parte superior del contenedor. Este ejemplo utiliza las regiones *CENTER* y *SOUTH*.

Clases Adaptadoras

Muchas de las interfaces de escucha de eventos, como *MouseListener* y *MouseMotionListener*, contienen varios métodos. No siempre es deseable declarar todos los métodos en una interfaz de escucha de eventos. Por ejemplo, una aplicación podría necesitar solamente el manejador *mouseClicked* de la interfaz *MouseListener*, o el manejador *mouseDragged* de la interfaz *MouseMotionListener*. La interfaz *WindowListener* especifica siete métodos manejadores de eventos de ventana. Para muchas de las interfaces de escucha de eventos que contienen varios métodos, los paquetes *java.awt.event* y *javax.swing.event* proporcionan clases adaptadoras de escucha de eventos.

Una **clase adaptadora** implementa a una interfaz y proporciona una implementación predeterminada (con un cuerpo vacío para los métodos) de todos los métodos en la interfaz.

A continuación, se muestran varias clases adaptadoras de *java.awt.event*, junto con las interfaces que implementan. Usted puede extender una clase adaptadora para heredar la implementación predeterminada de cada método, y en consecuencia sobrescribir sólo los métodos que necesite para manejar eventos.

Clase adaptadora de eventos en <i>java.awt.event</i>	Implementa a la interfaz
<i>ComponentAdapter</i>	<i>ComponentListener</i>
<i>ContainerAdapter</i>	<i>ContainerListener</i>
<i>FocusAdapter</i>	<i>FocusListener</i>
<i>KeyAdapter</i>	<i>KeyListener</i>
<i>MouseAdapter</i>	<i>MouseListener</i>
<i>MouseMotionAdapter</i>	<i>MouseMotionListener</i>
<i>WindowAdapter</i>	<i>WindowListener</i>

Ejemplo de como extender la clase adaptadora *MouseAdapter*

En la siguiente aplicación demostraremos cómo determinar el número de clics del ratón (es decir, la cuenta de clics) y cómo diferenciar los distintos botones del ratón. El componente de escucha de eventos en esta aplicación es un objeto de la clase interna *ManejadorClicRaton* que extiende a *MouseAdapter*, por lo que podemos declarar sólo el método *mouseClicked* que necesitamos en este ejemplo.

```
// Archivo "MarcoDetallesRaton.java"
// Demostración de los clics del ratón y cómo diferenciar los botones del mismo.
import java.awt.BorderLayout;
import java.awt.event.MouseAdapter;
import java.awt.event.MouseEvent;
import javax.swing.JFrame;
import javax.swing.JLabel;

public class MarcoDetallesRaton extends JFrame
{
    private String detalles; // String que se muestra en barraEstado
    private final JLabel barraEstado; // JLabel que aparece en la parte inferior de la
                                    // ventana

    // constructor establece String de la barra de título y registra componente de
    // escucha del ratón
    public MarcoDetallesRaton()
    {
        super("Clics y botones del raton");
        barraEstado = new JLabel("Haga clic en el raton");
        add(barraEstado, BorderLayout.SOUTH);
        addMouseListener(new ManejadorClicRaton()); // agrega el manejador
    }

    // clase interna para manejar los eventos del ratón
    private class ManejadorClicRaton extends MouseAdapter
    {
        // maneja evento de clic del ratón y determina cuál botón se oprimió
        @Override
        public void mouseClicked(MouseEvent evento)
        {
            int xPos = evento.getX(); // obtiene posición x del ratón en caso requerirse
            int yPos = evento.getY(); // obtiene posición y del ratón en caso requerirse

            detalles = String.format("Se hizo clic %d vez(vezes)",
                                    evento.getClickCount());

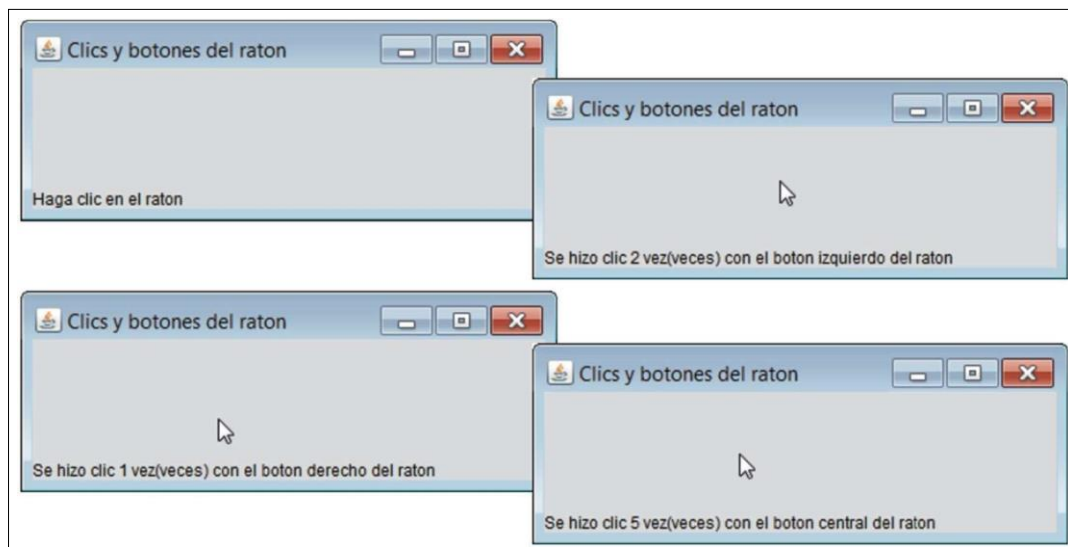
            if (evento.isMetaDown()) // botón derecho del ratón
                detalles += " con el boton derecho del raton";
            else if (evento.isAltDown()) // botón central del ratón
                detalles += " con el boton central del raton";
            else // botón izquierdo del ratón
                detalles += " con el boton izquierdo del raton";

            barraEstado.setText(detalles); // muestra mensaje en barraEstado
        } // fin de la clase MarcoDetallesRaton
    }
}
```

```
// PRUEBA DE Evento de Raton con Clase Adaptadora
// Prueba de MarcoDetallesRaton.
import javax.swing.JFrame;

public class App
{
    public static void main(String[] args) {
        MarcoDetallesRaton marcoDetallesRaton = new MarcoDetallesRaton();
        marcoDetallesRaton.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        marcoDetallesRaton.setSize(400, 150);
        marcoDetallesRaton.setVisible(true);
    }
} // fin de la clase DetallesRaton
```

Un ejemplo de lo mostrado por el programa si lo ejecutamos sería el siguiente:



Los métodos nuevos en este ejemplo son **getClickCount** que devuelve el número de clics del ratón y los métodos **isMetaDown** e **isAltDown** que determinan cuál botón del ratón oprimió el usuario. A continuación, se muestra el detalle de los mismos.

Método InputEvent	Descripción
isMetaDown()	Devuelve true cuando el usuario hace clic en el <i>botón derecho del ratón</i> , en un ratón con dos o tres botones. Para simular un clic con el botón derecho del ratón en un ratón con un botón, el usuario puede mantener oprimida la tecla <i>Meta</i> en el teclado y hacer clic con el botón del ratón.
isAltDown()	Devuelve true cuando el usuario hace clic con el <i>botón central del ratón</i> , en un ratón con tres botones. Para simular un clic con el botón central del ratón en un ratón con uno o dos botones, el usuario puede oprimir la tecla <i>Alt</i> en el teclado y hacer clic en el único botón o en el botón izquierdo del ratón, respectivamente.

Manejo de Eventos de Teclas

En esta sección presentamos la interfaz **KeyListener** para manejar eventos de teclas. Estos eventos se generan cuando se oprimen y sueltan las teclas en el teclado. Una clase que implementa a **KeyListener** debe proporcionar declaraciones para los métodos **keyPressed**, **keyReleased** y **keyTyped**, cada uno de los cuales recibe un objeto **KeyEvent** como argumento. La clase **KeyEvent** es una subclase de **InputEvent**.

- El método **keyPressed** es llamado en respuesta a la acción de oprimir cualquier tecla.
- El método **keyTyped** es llamado en respuesta a la acción de oprimir una tecla que no sea una tecla de acción. (Las teclas de acción son cualquier *tecla de dirección*, *Inicio*, *Fin*, *RePág*, *AvPág*, cualquier tecla de función, etc.).
- El método **keyReleased** es llamado cuando la tecla se suelta después de un evento **keyPressed** o **keyTyped**.

Ejemplo de uso de los métodos de KeyListener

La clase **MarcoDemoTeclas** implementa la interfaz **KeyListener**, por lo que los tres métodos se declaran en la aplicación. El constructor registra a la aplicación para manejar sus propios eventos de teclas, utilizando el método **addKeyListener**. Este método se declara en la clase **Component**, por lo que todas las subclases de **Component** pueden notificar a objetos **KeyListener** acerca de los eventos de teclas para ese objeto **Component**.

```
// Fig. 12.36: MarcoDemoTeclas.java
// Manejo de eventos de teclas.
import java.awt.Color;
import java.awt.event.KeyListener;
import java.awt.event.KeyEvent;
import javax.swing.JFrame;
import javax.swing.JTextArea;

public class MarcoDemoTeclas extends JFrame implements KeyListener {
    private String linea1 = ""; // primera línea del área de texto
    private String linea2 = ""; // segunda línea del área de texto
    private String linea3 = ""; // tercera línea del área de texto
    private JTextArea areaTexto; // área de texto para mostrar la salida

    // constructor de MarcoDemoTeclas
    public MarcoDemoTeclas()
    {
        super("Demostracion de los eventos de pulsacion de teclas");

        areaTexto = new JTextArea(10, 15); // establece el objeto JTextArea
        areaTexto.setText("Oprima cualquier tecla en el teclado...");
        areaTexto.setEnabled(false);
        areaTexto.setDisabledTextColor(Color.BLACK);
        add(areaTexto); // agrega el área de texto a JFrame

        addKeyListener(this); // permite al marco procesar los eventos de teclas
    }

    // maneja el evento de oprimir cualquier tecla
    @Override
    public void keyPressed(KeyEvent evento)
```

```

    {
        lineal = String.format("Tecla oprimida: %s",
                               KeyEvent.getKeyText(evento.getKeyCode())); // muestra la tecla oprimida
        establecerLineas2y3(evento); // establece las líneas de salida dos y tres
    }

    // maneja el evento de liberar cualquier tecla
    @Override
    public void keyReleased(KeyEvent evento)
    {
        lineal = String.format("Tecla liberada: %s",
                               KeyEvent.getKeyText(evento.getKeyCode())); // muestra la tecla liberada
        establecerLineas2y3(evento); // establece las líneas de salida dos y tres
    }

    // maneja el evento de oprimir una tecla de acción
    @Override
    public void keyTyped(KeyEvent evento)
    {
        lineal = String.format("Tecla oprimida: %s", evento.getKeyChar());
        establecerLineas2y3(evento); // establece las líneas de salida dos y tres
    }

    // establece las líneas de salida dos y tres
    private void establecerLineas2y3(KeyEvent evento)
    {
        linea2 = String.format("Esta tecla %s es una tecla de accion",
                               (evento.isActionKey() ? "" : "no "));

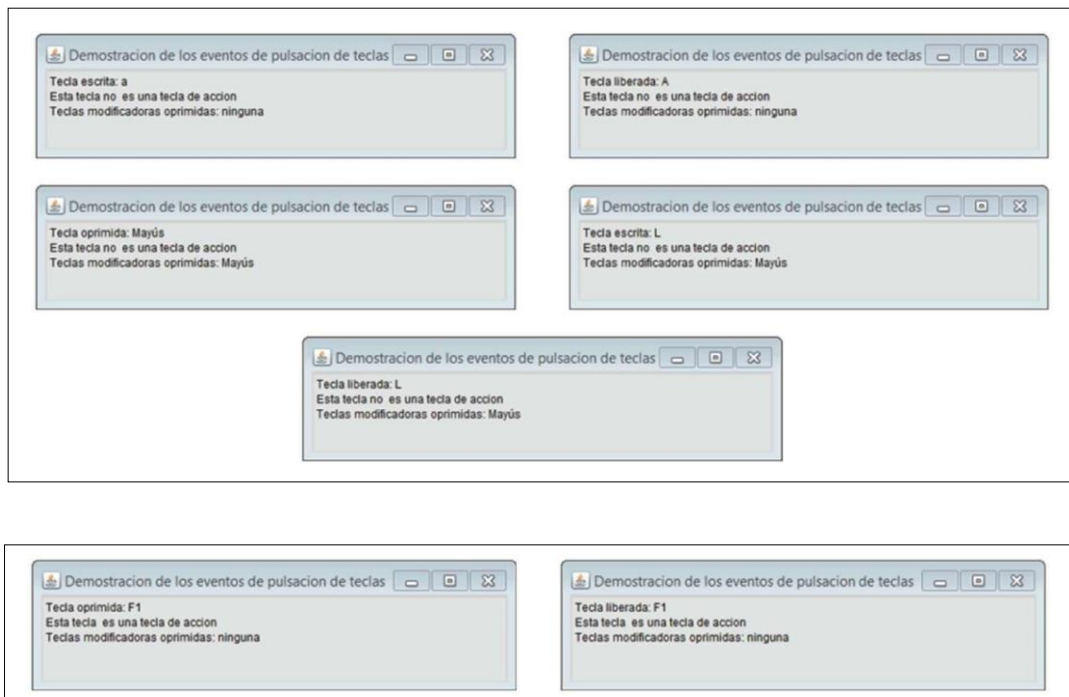
        String temp = KeyEvent.getModifiersExText(evento.getModifiersEx());
        linea3 = String.format("Teclas modificadoras oprimidas: %s",
                               (temp.equals("") ? "ninguna" : temp)); // imprime modificadoras
        areaTexto.setText(String.format("%s\n%s\n%s\n", lineal, linea2, linea3)); //
                                                                    // imprime tres líneas de texto
    }
} // fin de la clase MarcoDemoTeclas

// PRUEBA DE Evento de Tecla
// Prueba de MarcoDemoTeclas
import javax.swing.JFrame;

public class App
{
    public static void main(String[] args) {
        MarcoDemoTeclas marcoDemoTeclas = new MarcoDemoTeclas();
        marcoDemoTeclas.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        marcoDemoTeclas.setSize(350, 100);
        marcoDemoTeclas.setVisible(true);
    }
} // fin de la clase DemoTeclas

```

Un ejemplo de lo mostrado por el programa si lo ejecutamos sería el siguiente:



La clase **KeyEvent** mantiene un conjunto de constantes de códigos virtuales que representan a todas las teclas en el teclado. Estas constantes pueden compararse con el valor de retorno de **getKeyCode** para probar teclas individuales en el teclado. El valor devuelto por **getKeyCode** se pasa al método **getKeyText** de **KeyEvent**, el cual devuelve una cadena que contiene el nombre de la tecla que se oprimió. El método **keyTyped** utiliza el método **getKeyChar** de **KeyEvent** (el cual devuelve un valor **char**) para obtener el valor Unicode del carácter escrito.

Los tres métodos manejadores de eventos terminan llamando al método **establecerLineas2y3** y le pasan el objeto **KeyEvent**. Este método utiliza el método **isActionKey** de **KeyEvent** para determinar si la tecla en el evento fue una tecla de acción. Además, se hace una llamada al método **getModifiersEx** de **InputEvent** para determinar si se oprimió alguna tecla modificadora (como **Mayús**, **Alt** y **Ctrl**) cuando ocurrió el evento de tecla. El resultado de este método se pasa al método **getModifiersExText** de **KeyEvent**, el cual produce un objeto **String** que contiene los nombres de las teclas modificadoras que se oprimieron.

Administradores de Esquemas

Los **administradores de esquemas** ordenan los componentes de la GUI en un contenedor, para fines de presentación. Los programadores pueden usar los administradores de esquemas como herramientas básicas de distribución visual, en vez de determinar la posición y tamaño exactos de cada componente de la GUI. Esta funcionalidad permite al programador concentrarse en la apariencia general, y deja que el administrador de esquemas procese la mayoría de los detalles de la distribución visual. Todos los administradores de esquemas implementan la interfaz **LayoutManager** (en el paquete *java.awt*). El método **setLayout** de la clase **Container** toma un objeto que implementa a la interfaz **LayoutManager** como argumento.

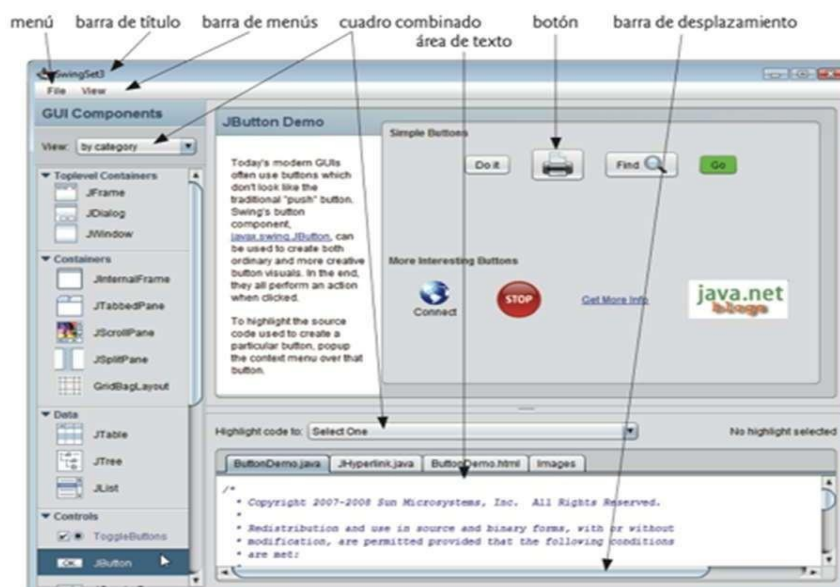
A continuación, se muestran los principales administradores de esquemas.

Administrador de esquemas	Descripción
FlowLayout	Es el predeterminado para <code>javax.swing.JPanel</code> . Coloca los componentes <i>secuencialmente (de izquierda a derecha)</i> en el orden en que se agregaron. También es posible especificar el orden de los componentes utilizando el método <code>add</code> de <code>Container</code> , el cual toma un objeto <code>Component</code> y una posición de índice entero como argumentos.
BorderLayout	Es el predeterminado para los objetos <code>JFrame</code> (y otras ventanas). Ordena los componentes en cinco áreas: NORTH, SOUTH, EAST, WEST y CENTER.
GridLayout	Ordena los componentes en filas y columnas

Adicionalmente a los administradores de esquemas, existen 2 formas para poder ordenar los componentes en una GUI:

- 1) **Posicionamiento absoluto:** esto proporciona el mayor nivel de control sobre la apariencia de una GUI. Al establecer el esquema de un objeto *Container* en *null*, podemos especificar la *posición absoluta* de cada componente de la GUI con respecto a la esquina superior izquierda del objeto *Container*, usando los métodos `setSize` y `setLocation` o `setBounds` de *Component*. Si hacemos esto, también debemos especificar el tamaño de cada componente de la GUI.
- 2) **Programación visual en un IDE:** los IDE proporcionan herramientas que facilitan la creación de GUI. Por lo general, cada IDE proporciona una herramienta de diseño de GUI que nos permite arrastrar y soltar componentes de GUI desde un cuadro de herramientas hacia un área de diseño. Después podemos posicionar, ajustar el tamaño de los componentes de la GUI y alinearlos según lo deseado. El IDE genera el código de Java que crea la GUI.

Uso de paneles para administrar esquemas más complejos



Las GUI complejas como la mostrada arriba de este párrafo requieren que cada componente se

coloque en una ubicación exacta. A menudo constan de varios paneles, en donde los componentes de cada panel se ordenan en un esquema específico. La clase *JPanel* extiende a *JComponent*, y *JComponent* extiende a la clase *Container*, por lo que todo *JPanel* es un *Container*. Por lo tanto, todo objeto *JPanel* puede tener componentes, incluyendo otros paneles, los cuales se adjuntan mediante el método `add` de *Container*.

En la aplicación que se muestra a continuación puede usarse un objeto *JPanel* para crear un esquema más complejo, en el cual se coloquen varios objetos *JButton* en la región *SOUTH* de un esquema *BorderLayout*.

```
// Archivo "MarcoPanel.java"
// Uso de un objeto JPanel para ayudar a distribuir los componentes.
import java.awt.GridLayout;
import java.awt.BorderLayout;
import javax.swing.JFrame;
import javax.swing.JPanel;
import javax.swing.JButton;

public class MarcoPanel extends JFrame
{
    private final JPanel panelBotones; // panel que contiene los botones
    private final JButton[] botones;

    // constructor sin argumentos
    public MarcoPanel()
    {
        super("Demostracion de Panel");
        botones = new JButton[5];
        panelBotones = new JPanel();
        panelBotones.setLayout(new GridLayout(1, botones.length));

        // crea y agrega los botones
        for (int cuenta = 0; cuenta < botones.length; cuenta++)
        {
            botones[cuenta] = new JButton("Boton " + (cuenta + 1));
            panelBotones.add(botones[cuenta]); // agrega el botón al panel
        }

        add(panelBotones, BorderLayout.SOUTH); // agrega el panel a JFrame
    }
} // fin de la clase MarcoPanel

// PRUEBA DE Panel para administrar esquema complejo
// Prueba de MarcoPanel
import javax.swing.JFrame;

public class App extends JFrame
{
    public static void main(String[] args) {
        MarcoPanel marcoPanel = new MarcoPanel();
        marcoPanel.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        marcoPanel.setSize(450, 200);
        marcoPanel.setVisible(true);
    }
} // fin de la clase DemoPanel
```

Un ejemplo de lo mostrado por el programa si lo ejecutamos sería el siguiente:



- 1) Una vez que el objeto *JPanel* llamado *panelBotones* se declara y se crea, se establece el esquema de *panelBotones* a un *GridLayout* con una fila y cinco columnas (hay cinco objetos *JButton* en el arreglo botones).
- 2) Se agregan los cinco objetos *JButton* en el arreglo al objeto *JPanel*.
- 3) Se agregan los botones directamente al objeto *JPanel* (la clase *JPanel* no tiene un panel de contenido, a diferencia de *JFrame*).
- 4) Se utiliza el objeto *BorderLayout* predeterminado de *JFrame* para agregar *panelBotones* a la región *SOUTH*. Esta región tiene la misma altura que los botones en *panelBotones*. Un objeto *JPanel* ajusta su tamaño de acuerdo con los componentes que contiene. A medida que se agregan más componentes, el objeto *JPanel* crece (de acuerdo con las restricciones de su administrador de esquemas) para dar cabida a esos nuevos componentes.
- 5) Si se ajusta el tamaño de la ventana se verá cómo el administrador de esquemas afecta al tamaño de los objetos *JButton*.

IV

ACTIVIDADES

Según el código visto en el marco teórico implementen tres ventanas que utilicen diferentes tipos de administradores de esquemas *FlowLayout*, *BorderLayout* y *GridLayout*.

Para la implementación de cada uno de los administradores, creen ventanas con títulos diferentes, agréguenles diferentes funcionalidades según los temas vistos en el marco teórico.

Hagan capturas de pantalla por cada implementación de administrador de esquema de tal forma que se evidencie cómo se organiza el contenido y las interacciones que se puede realizar en cada ventana.

V

EJERCICIOS

1. Investigue en el IDE de su preferencia cómo se puede utilizar herramientas visuales que

permitan crear componentes GUI Swing a través de operaciones “soltar y arrastrar” realizadas con el mouse de tal manera que sólo se tenga que agregar código a las clases y manejadores ya creados de manera automática por el IDE. Explique brevemente lo siguiente:

- a. Como es el proceso de creación de una GUI utilizando herramientas de “soltar y arrastrar” y haga capturas de pantalla que evidencien el mismo.
 - b. Cuáles serían algunas ventajas y desventajas de crear una GUI desde código y de hacerlo con una herramienta de “soltar y arrastrar”
2. Implemente una pequeña aplicación que simule el ingreso de datos para la compra de pasajes de una empresa de transporte terrestre. Para esto deberá utilizar todos los conceptos y componentes ya vistos. Una vez que se ingresen todos los datos del pasajero, se deberá mostrar un resumen de sus datos en un cuadro de Dialogo luego de presionar un botón. A continuación, se muestran cómo se utilizarían los componentes para ingresar un tipo de información específica:

Nombre de Componente	Propósito que cumplirá en la GUI de compra de pasaje
Etiquetas	Rotular que tipo de información se ingresara en cada componente
Campos de Texto	Ingreso de nombres, documento de identidad y fecha de viaje
Botón de Comando	Reiniciar o blanquear todos los componentes y mostrar diálogo
Casillas de Verificación	Indicar servicio opcional para pasajero (audifonos, manta, revistas)
Botones de Opción	Indicar si el pasajero quiere viajar en 1er o 2do piso
Cuadros combinados	Permitir al pasajero elegir un lugar de origen y de destino
Lista	Elegir entre 3 calidades de servicio (económico, standard, VIP)

VI

BIBLIOGRAFÍA

- Deitel Paul, Deitel Harvey. “COMO PROGRAMAR EN JAVA”, 9na. Edición, Edit. Prentice Hall, 2012.

RÚBRICA PARA LA CALIFICACIÓN DEL LABORATORIO

CRITERIO A SER EVALUADO		Nivel				
		A	B	C	D	NP
R1	Uso de estándares y buenas prácticas de programación	2	1.5	1	0.5	0
R2	Manejo y uso adecuado de los componentes Swing	4	2	1.5	0.5	0
R3	Informe detallado, bien redactado y ordenado con las respuestas y evidencias de las actividades.	6	-	-	-	0
R4	Resolución de los ejercicios donde se hace uso adecuado de componentes y conceptos para crear las GUI solicitadas según sea el caso. Mostrar evidencia suficiente.	8	-	-	-	0
Total		20				